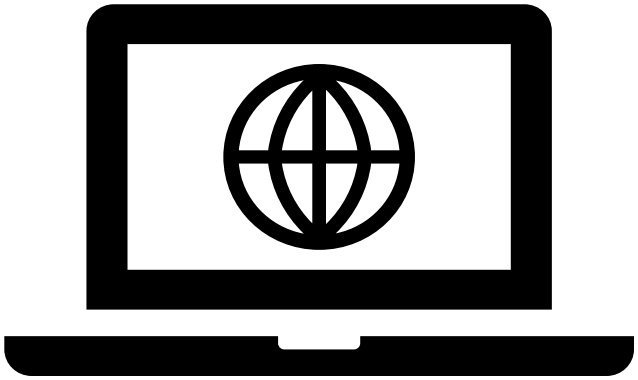
Sistemas Operacionais (SO)

Aula 03

Prof.: Nathan Saraiva



Roteiro

- **Estrutura de um SO**
  - Introdução;
  - Funções do núcleo;
  - Modo de acesso;
  - Rotinas do SO e *System Calls*;
  - Chamada a rotinas do SO;
  - Linguagem de comandos;
  - Ativação/Desativação do sistema;
  - Arquiteturas do núcleo.

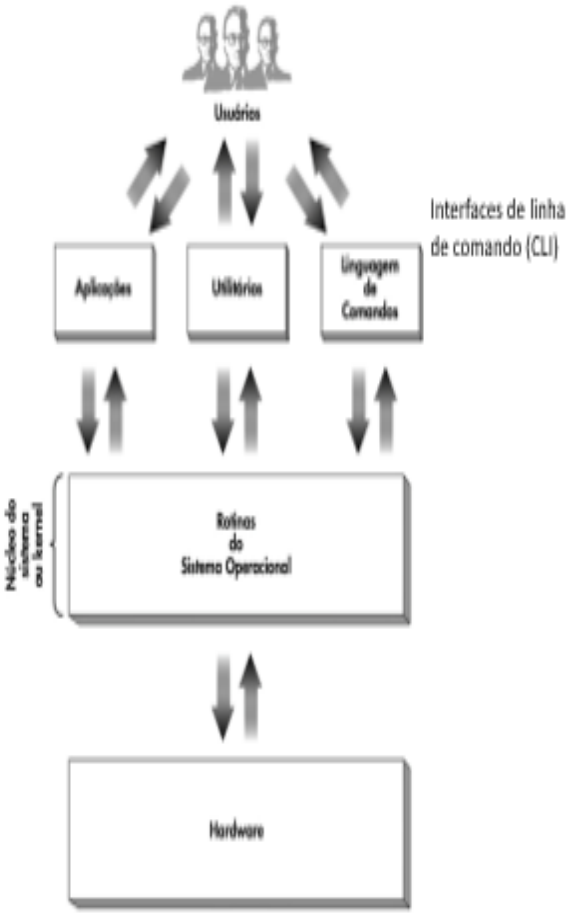
Introdução

- **Conceitos básicos**
  - O sistema operacional é formado por um conjunto de rotinas que oferece serviços aos usuários e às suas aplicações. Esse conjunto de rotinas é denominado núcleo do sistema, ou ***kernel***. A maioria dos sistemas operacionais é fornecida acompanhada de utilitários e linguagem de comandos, que são ferramentas de apoio ao usuário, porém não são parte do núcleo do sistema (MACHADO; MAIA, 2013).

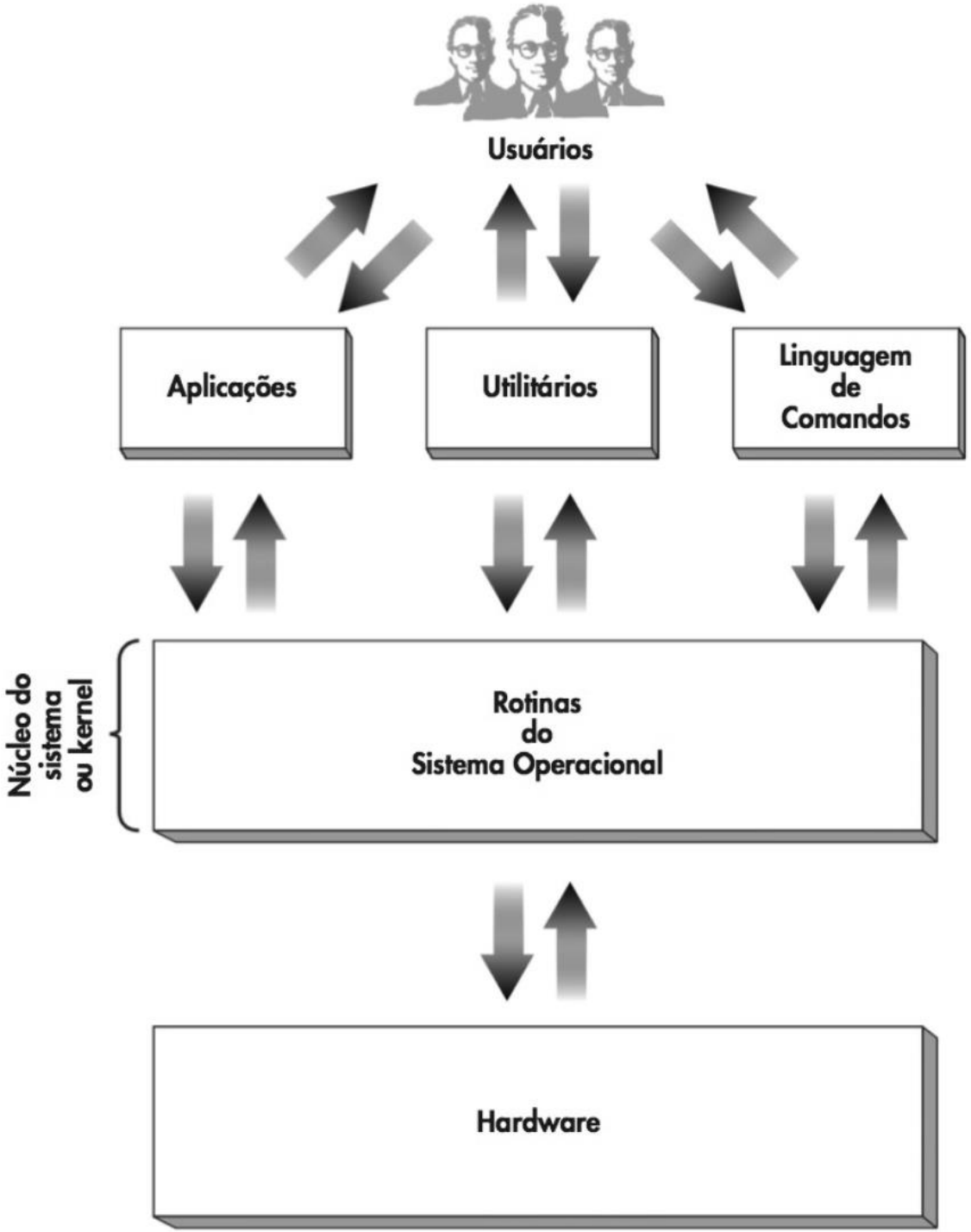
Introdução

- **Conceitos básicos**
  - Na Figura 1 é apresentado um modelo de camadas detalhando a estrutura do sistema operacional e suas interfaces.

Figura 1 – Estrutura do sistema operacional.



Fonte: (MACHADO; MAIA, 2013).



Interfaces de linha de comando (CLI)

Introdução

- **Conceitos básicos**
  - Há três maneiras dos usuários se comunicarem com o *kernel* do SO, conforme descrito abaixo.
    - Por meio das chamadas rotinas do sistema realizadas por aplicações;
    - Através de utilitários (gerenciamento e manutenção do sistema pelo usuário) como compiladores, montadores,desfragmentadores de disco, limpadores de disco, ferramentas de backup
    - Por intermédio de linguagem de comandos, ex: Bash, PowerShell

Introdução

- **Conceitos básicos**
  - Inicialmente serão apresentadas as funções do núcleo e os conceitos relativos à segurança, proteção do sistema, modos de acesso, rotinas do sistema, *system calls*, linguagem de comandos e ativação/desativação do sistema.

  - A compreensão da estrutura e do funcionamento de um sistema operacional <u>não é simples</u>. Diferentemente de uma aplicação convencional, com sequenciamento de início, meio e fim, as rotinas do sistema são executadas concorrentemente sem uma ordem predefinida, com base em eventos dissociados do tempo (eventos assíncronos). Muitos desses eventos estão relacionados ao *hardware* e a tarefas internas do próprio sistema operacional.

Funções do núcleo

- As principais funções do núcleo encontradas nos sistemas operacionais estão listadas a seguir:
  - tratamento de interrupções e exceções;
  - criação e eliminação de processos e *threads*;
  - sincronização e comunicação entre processos e *threads*;
  - escalonamento e controle dos processos e *threads*;
  - gerência de memória;
  - gerência do sistema de arquivos;
  - gerência de dispositivos de E/S;
  - suporte a redes locais e distribuídas;
  - contabilização do uso do sistema;
  - auditoria e segurança do sistema.

Funções do núcleo

- Uma das principais características da multiprogramação é permitir que vários programas compartilhem o processador. O sistema operacional deve ser responsável pelo controle da utilização da UCP(CPU), de forma a impedir que algum programa monopolize o seu uso inadequadamente.

Funções do núcleo

- Como vários programas ocupam a memória simultaneamente, cada usuário deve possuir uma área reservada onde seus dados e código são armazenados.

- O SO implementa mecanismos de **proteção**, de forma a preservar estas informações de maneira reservada. Caso um programa tente acessar uma posição de memória fora de sua área ocorrerá um erro indicando a violação de acesso.

- Para que diferentes programas tenham o direito de compartilhar uma mesma área de memória, o sistema operacional deve oferecer mecanismos para que a comunicação seja feita de forma sincronizada e controlada, evitando, desta forma, problemas de inconsistência.

Funções do núcleo

- De modo semelhante ao compartilhamento de memória, um disco pode armazenar arquivos de diferentes usuários. Mais uma vez o SO deve garantir a integridade e a confidencialidade dos dados, permitindo ainda que dois ou mais usuários possam ter acesso simultâneo ao mesmo arquivo.

- Para solucionar esses diversos problemas originados pelo ambiente multiprogramável, o sistema operacional deve implementar mecanismos de proteção que controlem o acesso concorrente aos diversos recursos do sistema.

Modo de acesso

- Uma preocupação que surge nos projetos de sistemas operacionais é a implementação de mecanismos de proteção ao núcleo do sistema e de acesso aos seus serviços.

- Caso uma aplicação, que tenha acesso ao núcleo, realize uma operação que altere sua integridade, todo o sistema poderá ficar **comprometido e inoperante**.

- Muitas das principais implementações de segurança de um sistema operacional utilizam um mecanismo presente no *hardware* dos processadores, conhecido como **modo de acesso**.

Modo de acesso

- Em geral, os processadores possuem dois modos de acesso: **modo usuário e modo *kernel***.

- Quando o processador trabalha no modo usuário, uma aplicação só pode executar instruções conhecidas como não privilegiadas, tendo acesso a um número reduzido de instruções, enquanto no modo *kernel* a aplicação pode ter acesso ao conjunto total de instruções do processador.

Modo de acesso

- As instruções privilegiadas **não** devem ser utilizadas de maneira indiscriminada pelas aplicações, pois isso poderia ocasionar sérios problemas à integridade do sistema.

- Ex: Suponha que uma aplicação atualize um arquivo em disco. O programa, por si só, não deve especificar diretamente as instruções que acessam seus dados no disco. Como o disco é um recurso compartilhado, sua utilização deverá ser gerenciada unicamente pelo SO, evitando que a aplicação possa ter acesso a qualquer área do disco sem autorização, o que poderia comprometer a segurança e a integridade do sistema de arquivos.

Modo de acesso

- **Curiosidade**
  - Como visto, fica claro que existem certas instruções que só devem ser executadas pelo sistema operacional ou sob sua supervisão, **impedindo**, assim, a ocorrência de problemas de segurança e integridade do sistema.

  - As instruções privilegiadas só podem ser executadas quando o modo de acesso do processador encontra-se em ***kernel*** *(bits no registrador)*, caso contrário o *hardware* irá impedir a execução da instrução. As instruções não privilegiadas são as que não oferecem risco ao sistema e podem ser executadas em modo não privilegiado, ou seja, modo usuário.

Modo de acesso

- Outro exemplo do uso dos modos de acessos é a proteção do próprio núcleo do sistema residente na memória principal.

- Caso uma aplicação tenha acesso a áreas de memória onde está carregado o SO, um programador mal-intencionado ou um erro de programação poderia gravar nesta área, **violando o sistema**. Com o mecanismo de modo de acesso, para uma aplicação escrever numa área onde resida o sistema operacional o programa deve estar sendo executado no modo *kernel*.

Rotinas do SO e *System Calls*

- As rotinas do sistema operacional compõem o núcleo do sistema, oferecendo serviços aos usuários e suas aplicações.

- Todas as funções do núcleo são implementadas por rotinas do sistema que necessariamente possuem em seu código instruções privilegiadas.

- A partir desta condição, para que estas rotinas possam ser executadas o processador deve estar obrigatoriamente em **modo *kernel***, o que exige a implementação de mecanismos de proteção para garantir a confiabilidade do sistema.

Rotinas do SO e *System Calls*

- Todo o controle de execução de rotinas do sistema operacional é realizado pelo mecanismo conhecido como ***system call*.**

- Toda vez que uma aplicação desejar chamar uma rotina do SO, o mecanismo de *system call* é ativado.

- Inicialmente, o sistema operacional verificará se a aplicação possui privilégios necessários para executar a rotina desejada. Em caso negativo, o SO impedirá o desvio para a rotina do sistema, sinalizando ao programa chamador que a operação não é possível. Este é um mecanismo de proteção por *software*, no qual o SO garante que as aplicações só poderão executar rotinas do sistema que estão **previamente autorizadas**.

Rotinas do SO e *System Calls*

- Considerando que a aplicação possua o devido privilégio para chamar a rotina do sistema desejada, o SO primeiramente salva o conteúdo corrente dos registradores, troca o modo de acesso do processador de usuário para kernel e realiza o desvio para a rotina alterando o registrador PC com o endereço da rotina chamada (ver Figura 2).

- Ao término da execução da rotina do sistema, o modo de acesso é alterado de *kernel* para usuário e o contexto dos registradores restaurados para que a aplicação continue a execução a partir da instrução que chamou a rotina do sistema.

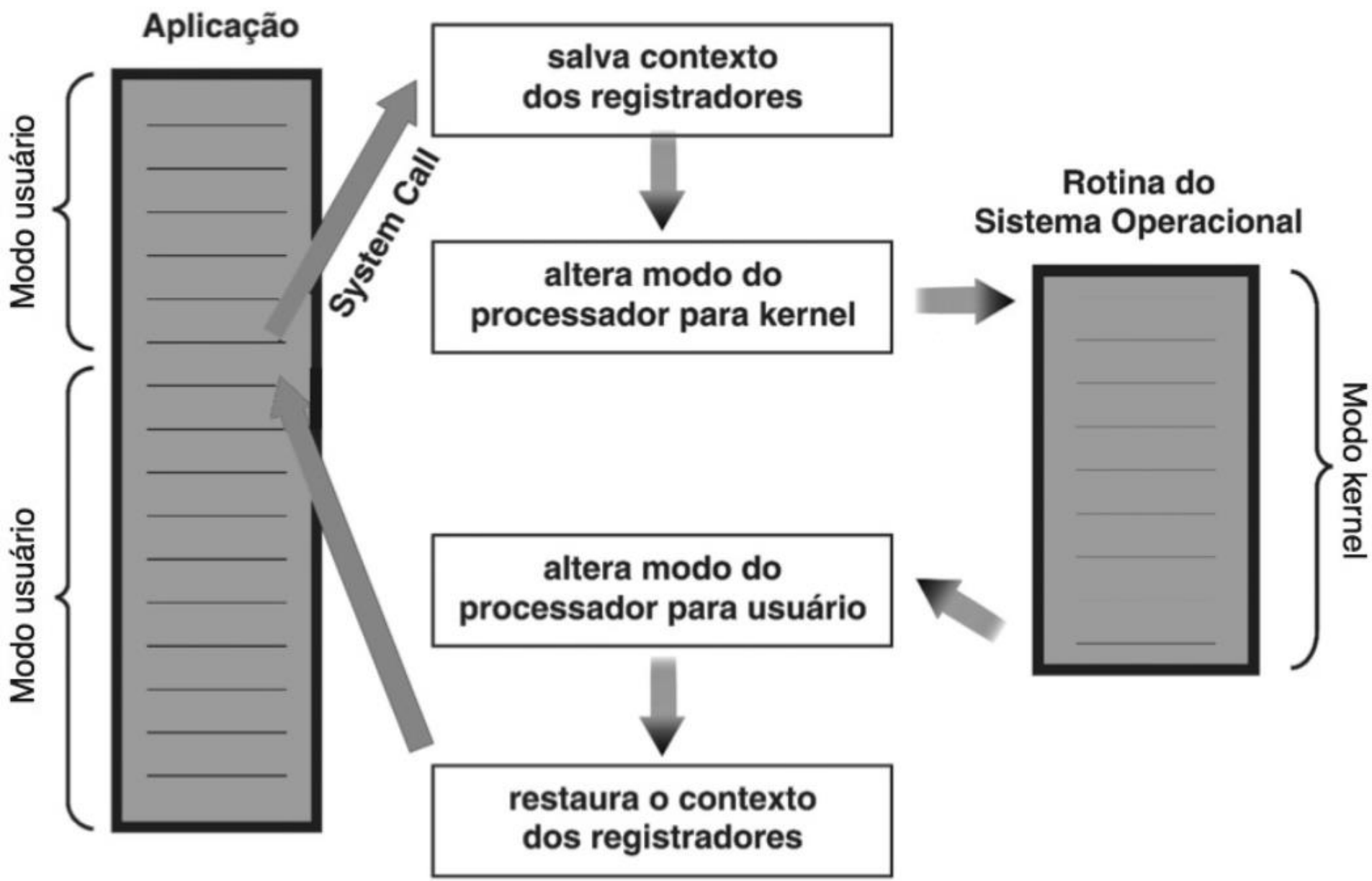


Figura 2 – Detalhamento de chamada a uma rotina do sistema.

Rotinas do SO e *System Calls*

- Uma aplicação sempre deve executar com o processador em modo usuário. Caso uma aplicação tente executar diretamente uma instrução privilegiada sem ser por intermédio de uma chamada à rotina do sistema, um mecanismo de proteção por *hardware* garantirá a segurança do sistema, impedindo a operação.

Chamada a Rotinas do Sistema Operacional

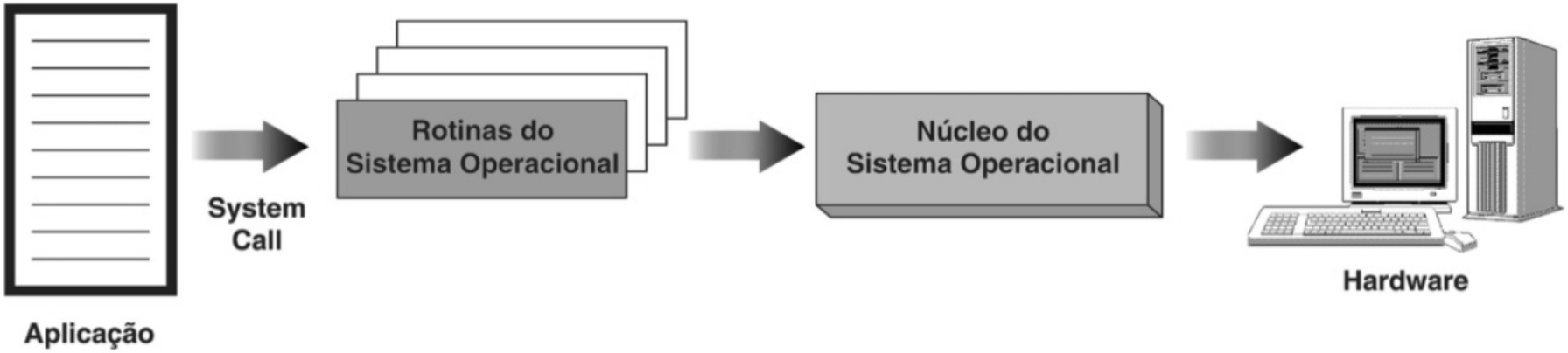
- As rotinas do sistema e o mecanismo de *system call* podem ser entendidos como uma porta de entrada para o <u>núcleo do sistema operacional e a seus serviços</u>.

- Sempre que uma aplicação desejar algum serviço do sistema, deve ser realizada uma chamada a uma de suas rotinas através de uma ***system call*** (ver Figura 3).

- Por intermédio dos parâmetros fornecidos na *system call*, a solicitação é processada e uma resposta é retornada à aplicação juntamente com um estado de conclusão indicando se houve algum erro.

Chamada a Rotinas do Sistema Operacional

Figura 3 – Chamada genérica a uma rotina do sistema.



Fonte: (MACHADO; MAIA, 2013).

Chamada a Rotinas do Sistema Operacional

- A maioria dos programadores e usuários desconhece os detalhes envolvidos, por exemplo, em um simples comando de leitura a um arquivo utilizando uma linguagem de alto nível.

- De forma simplificada, o comando da linguagem de alto nível é convertido pelo compilador para uma chamada a uma rotina específica que, quando executada, verifica a ocorrência de erros e retorna os dados ao programa de forma transparente ao usuário.

- As rotinas do sistema podem ser divididas por grupos de função, conforme é apresentado na Tabela 1.

Chamada a Rotinas do Sistema Operacional

Tabela 1 – Funções das *system calls*.

Funções	System calls
Gerência de processos e *threads*.	Criação e eliminação de processos e *threads*. Alteração das características de processos e *threads*. Sincronização e comunicação entre processos e *threads*. Obtenção de informações sobre processos e *threads*.
Gerência de memória.	Alocação e desalocação de memória.
Gerência do sistema de arquivos.	Criação e eliminação de arquivos e diretórios. Alteração das características de arquivos e diretórios. Abertura e fechamento de arquivos. Leitura e gravação em arquivos. Obtenção de informações sobre arquivos e diretório.
Gerência de dispositivos.	Alocação e desalocação de dispositivos. Operações de entrada/saída em dispositivos. Obtenção de informações sobre dispositivos.

Fonte: (MACHADO; MAIA, 2013).

Linguagem de Comandos

- A linguagem de comandos, ou linguagem de controle, permite que o usuário se comunique de uma forma simples com o sistema operacional, capacitando-o a executar diversas tarefas específicas do sistema como criar, ler ou eliminar arquivos, consultar diretórios ou verificar a data e a hora armazenadas no sistema.

- Dessa forma, o usuário dispõe de uma interface direta com o sistema operacional. A Tabela 2 apresenta exemplos de comandos disponíveis no sistema *Microsoft Windows* e em distribuições baseada no *Unix*.

- Cada SO possui a sua linguagem de comandos como, por exemplo, a DCL (*Digital Command Language*) utilizada no OpenVMS, JCL (*Job Control Language*) no MVS da IBM e os comandos do *shell* disponíveis nos diversos sistemas *Unix*.

Linguagem de Comandos

Tabela 2 – Exemplos de comandos.

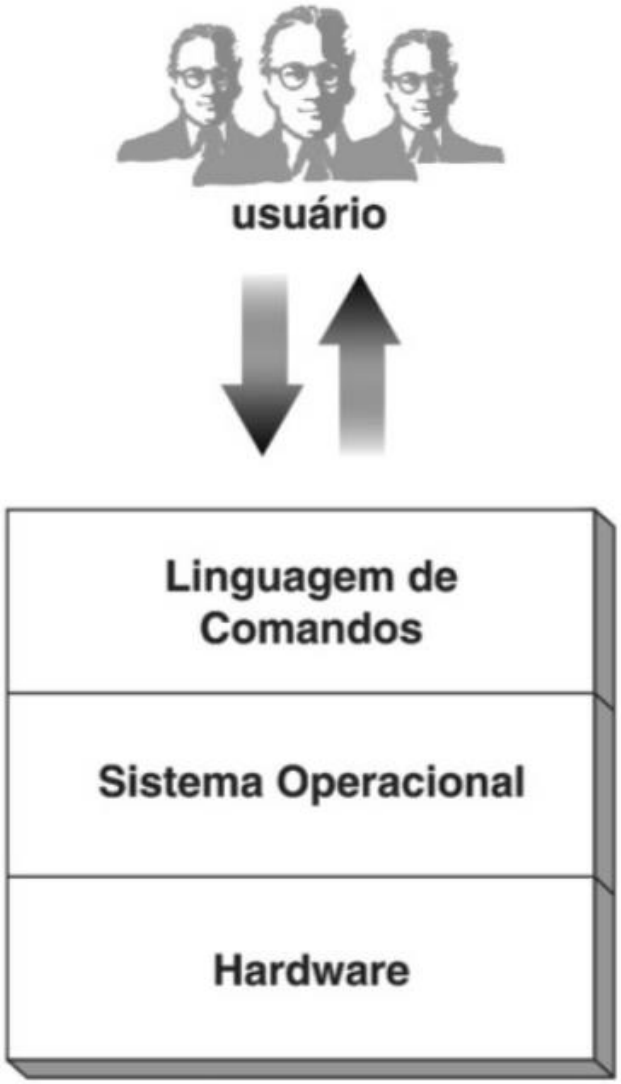
Comando – *Windows/Unix*	**Descrição**
dir/ls	Lista o conteúdo de um diretório
cd em ambos	Altera o diretório *default*
type/cat	Exibe o conteúdo de um arquivo
del/rm	Elimina arquivos
mkdir em ambos	Cria um diretório
ver/hostnamectl	Mostra a versão do sistema operacional

Fonte: (MACHADO; MAIA, 2013).

Linguagem de Comandos

- Cada comando, depois de digitado pelo usuário, é interpretado pelo *shell* ou interpretador de comandos, que verifica a sintaxe do comando, faz chamadas a rotinas do sistema e apresenta um resultado ou uma mensagem informativa (ver Figura 5a).

- Em geral, o interpretador de comandos não faz parte do núcleo do sistema operacional, possibilitando maior flexibilidade na criação de diferentes linguagens de controle para um mesmo sistema. Por exemplo, o *Unix* oferece, basicamente, três interpretadores de comandos: *Bourne Shell*, *C Shell* e *Korn Shell*.


usuário
Linguagem de Comandos
Sistema Operacional
Hardware

(a)

usuário
Interface Gráfica
Sistema Operacional
Hardware

(b)

Linguagem de Comandos

- Na maioria dos sistemas operacionais, as linguagens de comandos evoluíram no sentido de permitir a interação mais amigável com os usuários, utilizando interfaces gráficas como janelas e ícones, a exemplo dos sistemas *MS Windows*. Na maioria dos casos, a interface gráfica é apenas mais um nível de abstração entre o usuário e os serviços do sistema operacional (ver Figura 5b).

- As linguagens de comandos são poderosas a ponto de oferecer a possibilidade de criar programas com estruturas de decisão e iteração. Esses programas nada mais são do que uma sequência de comandos armazenados em um arquivo texto, denominados arquivos de comandos, arquivos *batch* ou *shell scripts*, que podem ser executados sempre que necessário. Uma das principais vantagens no uso de arquivos de comandos é possibilitar a automatização de diversas tarefas ligadas à gerência do sistema.

Ativação/Desativação do Sistema

- Inicialmente, quando um computador é ligado não há sistema operacional carregado na memória da máquina.

- Em geral, o sistema operacional reside em um disco rígido, podendo também estar armazenado em outros dispositivos de memória secundária, como CD ou DVD. Os componentes do sistema operacional devem ser carregados para a memória principal toda vez que o computador é ligado por intermédio de um procedimento denominado ativação do sistema ou ***boot***.

- O procedimento de ativação se inicia com a execução de um programa chamado gerenciador de inicialização (*boot loader*, em Inglês), que se localiza em um endereço fixo de uma memória ROM da máquina. Este programa chama a execução de outro programa conhecido como POST (*Power-On Self Test*), que identifica possíveis problemas de *hardware* no equipamento.

Ativação/Desativação do Sistema

- Após esta fase, o procedimento de ativação verifica se há no sistema computacional algum dispositivo de armazenamento onde haja um SO residente. Caso nenhum dispositivo seja encontrado, uma mensagem de erro é apresentada e o procedimento de ativação é interrompido.

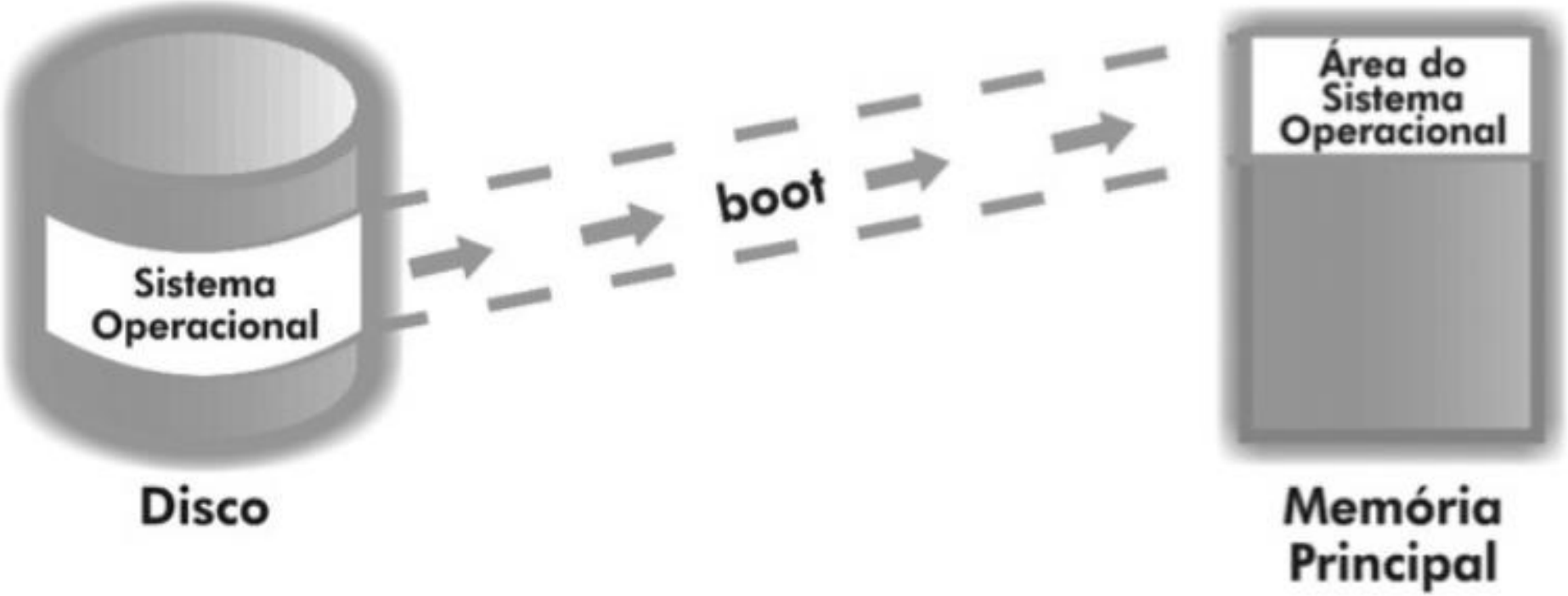
- Se um dispositivo com o SO é encontrado, um conjunto de instruções é carregado para memória e localizado em um bloco específico do dispositivo conhecido como setor de *boot* (boot sector, em Inglês). A partir da execução deste código, o sistema operacional é finalmente carregado para a memória principal. Além da carga, a ativação do sistema também consiste na execução de arquivos de inicialização onde são especificados procedimentos de customização e configuração de *hardware* e *software* específicos para cada ambiente (ver a Figura 6).

Ativação/Desativação do Sistema

- Na maioria dos sistemas também existe o processo de desativação(*shutdown*, em Inglês). Este procedimento permite que as aplicações e componentes do sistema operacional sejam desativados ordenadamente, garantindo, desta forma, sua integridade.

Figura 6 – Ativação do sistema.

Fonte: (MACHADO; MAIA, 2013).


Sistema
Operacional
Disco
boot
Área do
Sistema
Operacional
Memória
Principal

Arquiteturas do Núcleo

- O projeto de um SO é bastante complexo e deve atender a diversos requisitos, algumas vezes conflitantes, como confiabilidade, portabilidade, fácil manutenção, flexibilidade e desempenho. O projeto do sistema irá depender muito da arquitetura do *hardware* a ser usado e do tipo de sistema que se deseja construir: *batch*, tempo compartilhado, monousuário ou multiusuário, tempo real etc.

- Os primeiros sistemas operacionais foram desenvolvidos integralmente em *assembly*, e o código possuía cerca de um milhão de instruções (IBM OS/360). Com a evolução dos sistemas e o aumento do número de linhas de código para algo perto de 20 milhões (MULTICS), técnicas de programação modular foram incorporadas ao projeto, além de linguagens de alto nível, como PL/I e *Algol*. Nos sistemas operacionais atuais, o número de linhas de código pode chegar a mais de 40 milhões (*Windows* 2000), sendo que grande parte do código é escrita em Linguagem C/C++.

Arquiteturas do Núcleo

- Além de facilitar o desenvolvimento e a manutenção do sistema, a utilização de linguagens de alto nível permite uma maior portabilidade, ou seja, que o sistema operacional seja facilmente alterado em outra arquitetura de *hardware*. Uma desvantagem do uso de linguagens de alto nível em relação à programação *assembly* é a perda de desempenho. Por isso, as partes críticas do sistema, como os *device drivers*, o escalonador e as rotinas de tratamento de interrupções, são desenvolvidas em *assembly*.

- Uma tendência no projeto de sistemas operacionais modernos é a utilização de técnicas de orientação a objetos, o que leva para o projeto do núcleo do sistema todas as vantagens deste modelo de desenvolvimento de *software*. Existe uma série de vantagens na utilização de programação a objetos no projeto e na implementação de sistemas operacionais.

Arquiteturas do Núcleo

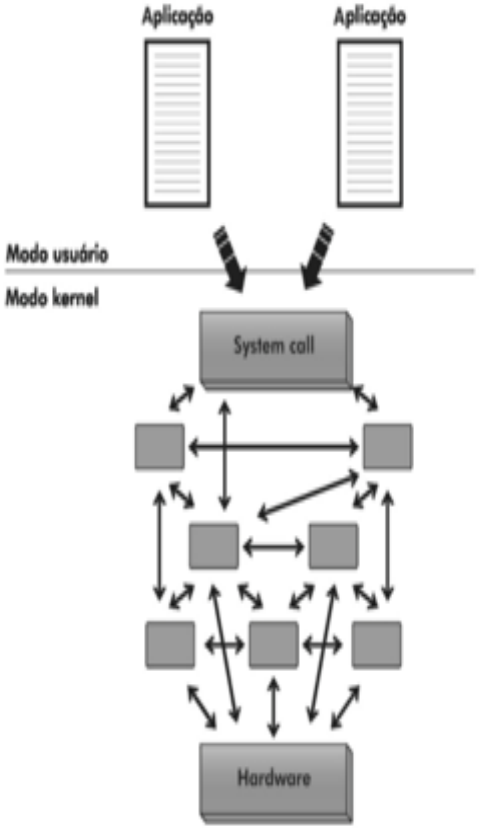
- A seguir, os principais benefícios são apresentados:
  - melhoria na organização das funções e recursos do sistema;
  - redução no tempo de desenvolvimento;
  - maior facilidade na manutenção e extensão do sistema;
  - facilidade de implementação do modelo de computação distribuída.

- A estrutura do núcleo do SO, ou seja, a maneira como o código do sistema é organizado e o inter-relacionamento de seus diversos componentes, pode variar conforme a concepção do projeto. A seguir serão abordadas as principais arquiteturas dos sistemas operacionais: arquitetura monolítica, arquitetura de camadas, máquina virtual e arquitetura micronúcleo(*microkernel*, em Inglês).

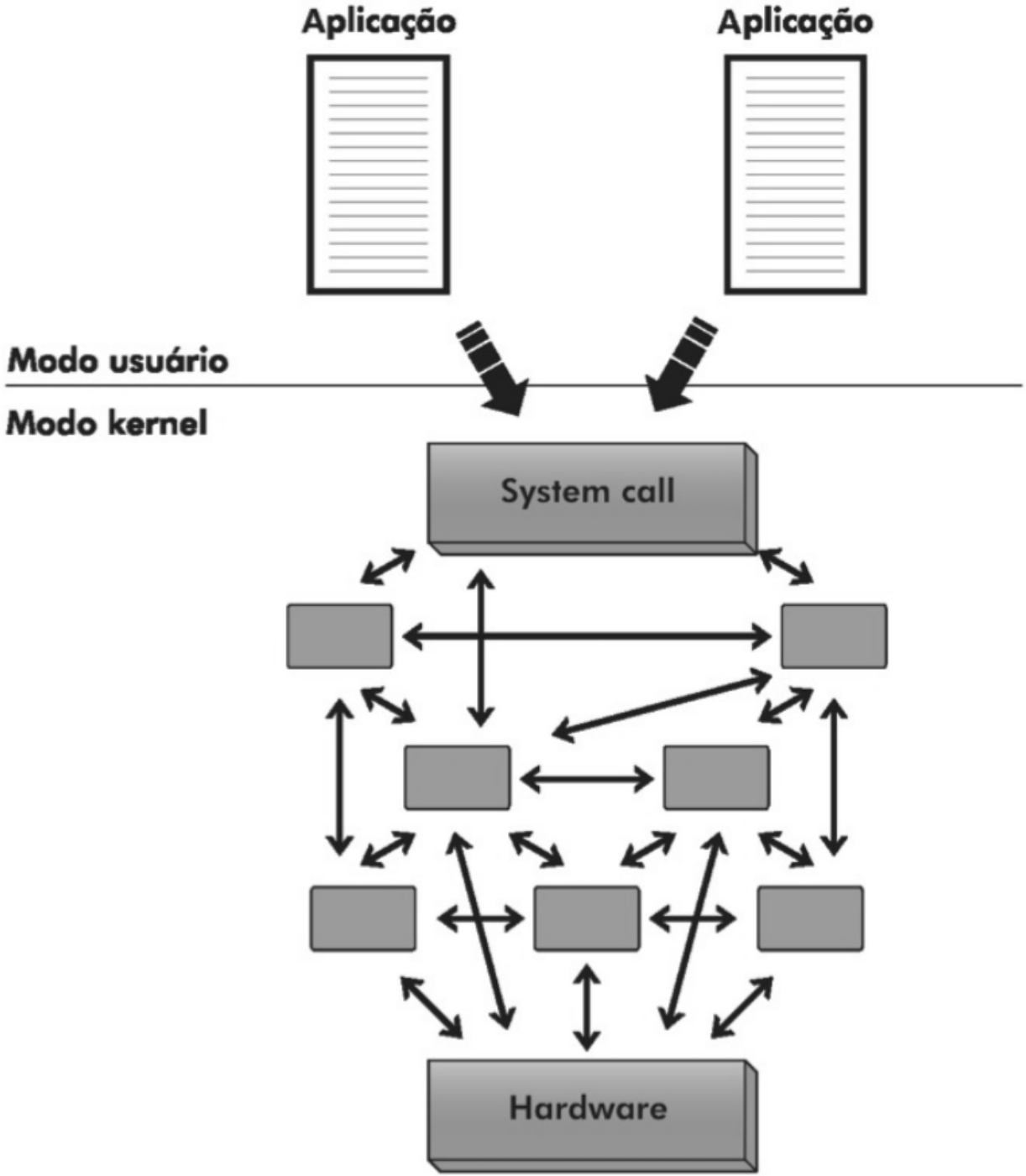
Arquiteturas do Núcleo

- **Arquitetura Monolítica**
  - A arquitetura monolítica pode ser comparada com uma aplicação formada por vários módulos que são compilados separadamente e depois integrados, formando um grande e único programa executável, onde os módulos podem interagir livremente (ver Figura 7).

Figura 7 – Arquitetura monolítica genérica.



Fonte: (MACHADO; MAIA, 2013).


Aplicação
Aplicação
Modo usuário
Modo kernel
System call
Hardware

Arquiteturas do Núcleo

- **Arquitetura Monolítica**
  - Curiosidade
    - Os primeiros sistemas operacionais foram desenvolvidos com base neste modelo, o que tornava seu desenvolvimento e, principalmente, sua manutenção bastante difíceis. Devido a sua simplicidade e bom desempenho, a estrutura monolítica foi adotada no projeto do MS-DOS e nos primeiros sistemas *Unix*.

Arquiteturas do Núcleo

- **Arquitetura de Camadas**
  - Com o aumento da complexidade e do tamanho do código dos sistemas operacionais, técnicas de programação estruturada e modular foram incorporadas ao seu projeto. Na arquitetura de camadas, o sistema é dividido em níveis sobrepostos. Cada camada oferece um conjunto de funções que podem ser utilizadas apenas pelas camadas superiores.

  - O primeiro sistema com base nesta abordagem foi o sistema THE (*Technische Hogeschool Eindhoven*), construído por Dijkstra na Holanda em 1968 e que utilizava seis camadas.

  - Posteriormente, os sistemas MULTICS e *OpenVMS* também implementaram o conceito de camadas, sendo estas concêntricas e em quantidade de quatro (ver Figura 8). Neste tipo de implementação, as camadas mais internas são mais privilegiadas que as mais externas.

Arquiteturas do Núcleo

- **Arquitetura de Camadas**
  - Com o aumento da complexidade e do tamanho do código dos sistemas operacionais, técnicas de programação estruturada e modular foram incorporadas ao seu projeto. Na arquitetura de camadas, o sistema é dividido em níveis sobrepostos. Cada camada oferece um conjunto de funções que podem ser utilizadas apenas pelas camadas superiores.

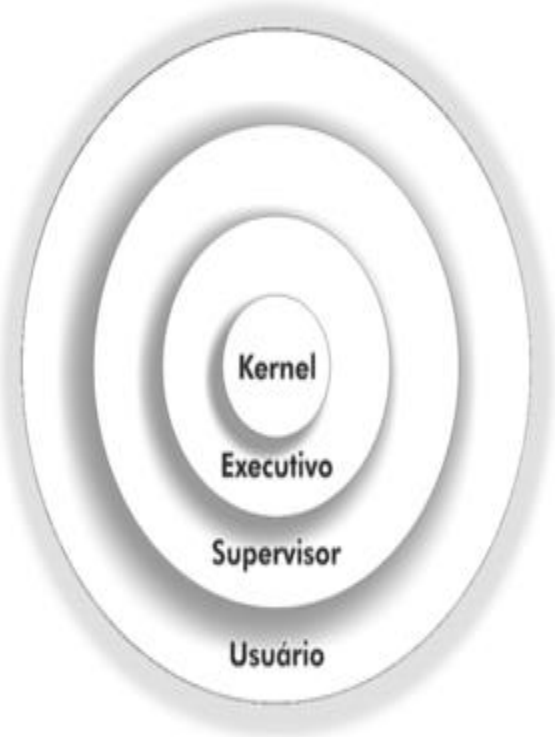
  - O primeiro sistema com base nesta abordagem foi o sistema THE (*Technische Hogeschool Eindhoven*), construído por Dijkstra na Holanda em 1968 e que utilizava seis camadas.

  - Posteriormente, os sistemas MULTICS e *OpenVMS* também implementaram o conceito de camadas, sendo estas concêntricas e em quantidade de quatro (ver Figura 8). Neste tipo de implementação, as camadas mais internas são mais privilegiadas que as mais externas.

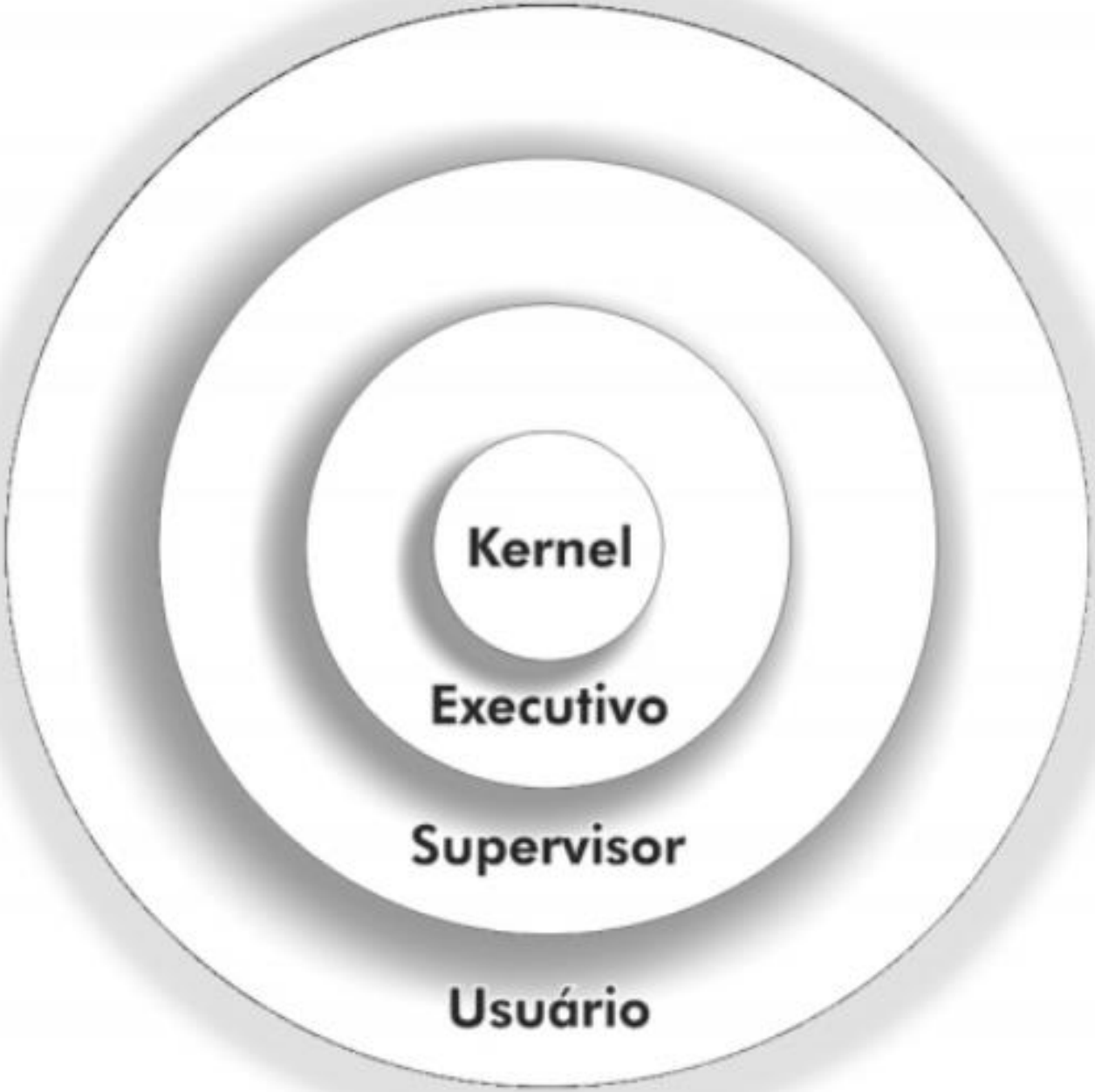
Arquiteturas do Núcleo

- **Arquitetura de Camadas**

Figura 8 – Arquitetura de camadas genérica.



Fonte: (MACHADO; MAIA, 2013).


Kernel
Executivo
Supervisor
Usuário

Arquiteturas do Núcleo

- **Arquitetura de Camadas**
  - A vantagem da estruturação em camadas é isolar as funções do sistema operacional, facilitando sua manutenção e depuração, além de criar uma hierarquia de níveis de modos de acesso, protegendo as camadas mais internas. Uma desvantagem para o modelo de camadas é o desempenho. Cada nova camada implica uma mudança no modo de acesso. Por exemplo, no caso do OpenVMS, para se ter acesso aos serviços oferecidos pelo *kernel* é preciso passar por três camadas ou três mudanças no modo de acesso.

Arquiteturas do Núcleo

- **Arquitetura de Camadas**
  - Curiosidade
    - Atualmente, a maioria dos sistemas comerciais utiliza o modelo de duas camadas, onde existem os modos de acesso usuário (não privilegiado) e *kernel* (privilegiado). A maioria das versões do *Unix* e o *Windows* está baseada neste modelo.

Arquiteturas do Núcleo

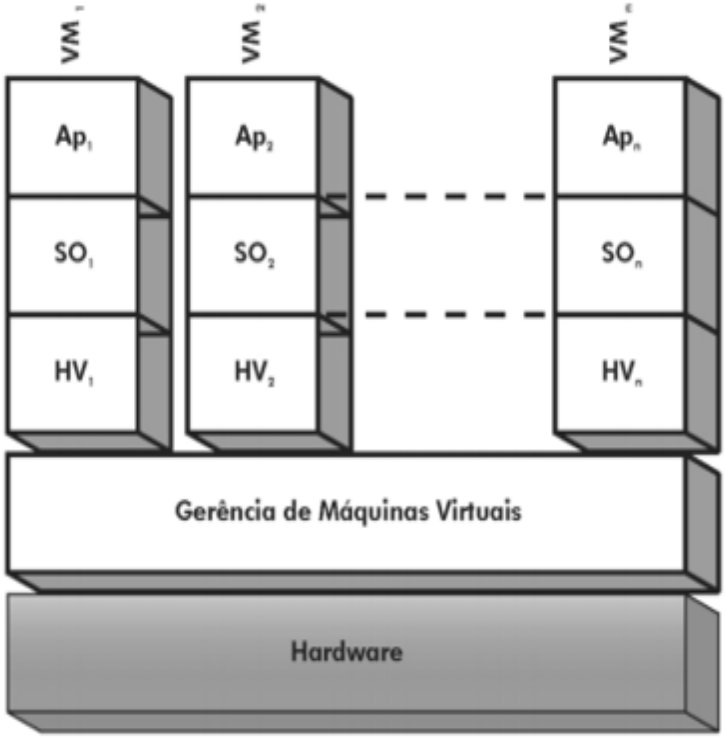
- **Máquina Virtual**
  - Um sistema computacional é formado por níveis, onde a camada de nível mais baixo é o *hardware*. Acima desta camada encontramos o sistema operacional que oferece suporte para as aplicações, como visto na Figura 1. O modelo de Máquina Virtual (*Virtual Machine* (VM), em Inglês), cria um nível intermediário entre o *hardware* e o sistema operacional, denominado gerência de máquinas virtuais (ver Figura 9).

  - Este nível cria diversas máquinas virtuais independentes, onde cada uma oferece uma cópia virtual do *hardware*, incluindo os modos de acesso, interrupções, dispositivos de E/S etc.

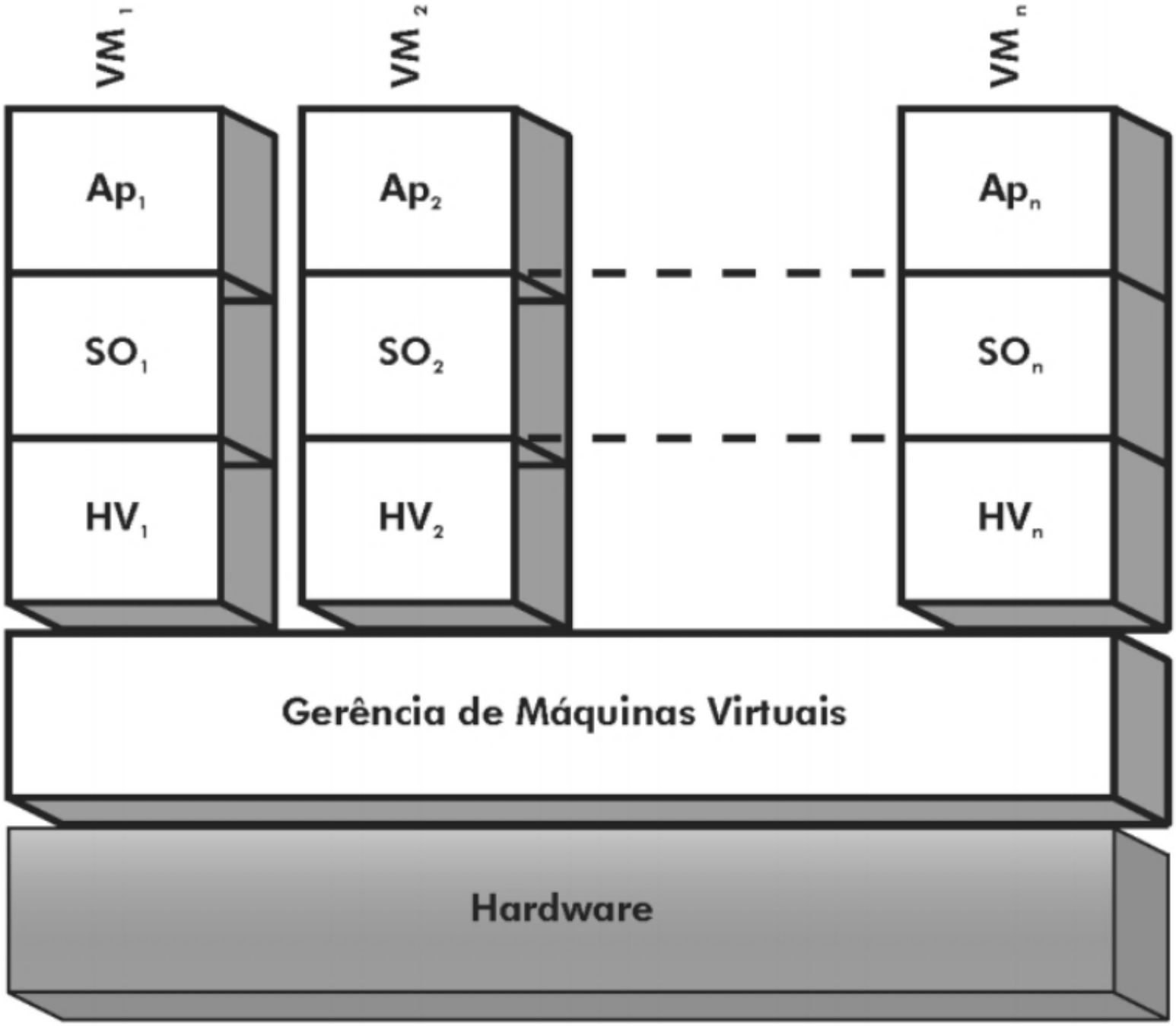
Arquiteturas do Núcleo

- **Máquina Virtual**

Figura 9 – Máquina virtual genérica.



Fonte: (MACHADO; MAIA, 2013).


VM 1
VM 2
VM n
Ap1
Ap2
Apn
SO1
SO2
SOn
HV1
HV2
HVn
Gerência de Máquinas Virtuais
Hardware

Arquiteturas do Núcleo

- **Máquina Virtual**
  - Como cada máquina virtual é independente das demais, é possível que cada VM tenha seu próprio SO e que seus usuários executem suas aplicações como se todo o computador estivesse dedicado a cada um deles.

  - Na década de 1960, a IBM implementou este modelo no sistema VM/370, permitindo que aplicações *batch*, originadas de antigos sistemas OS/360, e aplicações de tempo compartilhado pudessem conviver na mesma máquina de forma transparente a seus usuários e aplicações.

Arquiteturas do Núcleo

- **Máquina Virtual**
  - Este modelo cria o isolamento total entre cada VM, oferecendo grande segurança para cada máquina virtual. Se, por exemplo, uma VM executar uma aplicação que comprometa o funcionamento do seu SO, as demais máquinas virtuais não sofrerão qualquer problema. Apesar do isolamento das aplicações, as máquinas virtuais não estão livres de problemas, como, por exemplo, vírus e erros de *software* do sistema operacional ou aplicações da VM.

Arquiteturas do Núcleo

- **Máquina Virtual**
  - Existem diversas aplicações para a utilização de máquinas virtuais, tais como:
    - Portabilidade de código;
    - Consolidação de servidores;
    - Aumento da disponibilidade;
    - Facilidade de escalabilidade e balanceamento de carga;
    - Facilidade no desenvolvimento de *software*.

Arquiteturas do Núcleo

- **Máquina Virtual**
  - Portabilidade de código
    - Um programa executável, em linguagem de máquina, precisa de uma plataforma de *hardware* específica para ser executado, tornando-o dependente desse ambiente. Se o programa for gerado em uma linguagem intermediária e uma VM fizer a tradução dos comandos para a plataforma onde o programa está sendo executado, a aplicação pode ser portada para qualquer ambiente sem a necessidade de se reescrever o programa.

    - A máquina virtual *Java* (*Java Virtual Machine* – JVM) funciona dessa forma. O compilador *Java* gera um código intermediário, chamado *bytecode*, que é interpretado pela JVM e convertido para o ambiente onde está sendo executado. Dessa forma, o programa é desenvolvido apenas uma vez e pode ser processado em qualquer *hardware* ou sistema operacional, bastando que exista uma JVM para tal.

Arquiteturas do Núcleo

- **Máquina Virtual**
  - <u>Consolidação de servidores</u>
    - É muito comum que os servidores de uma empresa funcionem com apenas parte de sua capacidade total de processamento, ou seja, existe a subutilização da UCP, memória principal e dispositivos de E/S dos sistemas computacionais.

    - A ideia da consolidação é poder executar diversas aplicações que estariam sendo processadas em diferentes servidores, cada qual com o seu *hardware* e sistema operacional, em uma única máquina, utilizando uma VM para cada uma das aplicações. Dessa forma, a consolidação permite a melhor utilização dos recursos computacionais e, além disso, também permite a redução de custos de infraestrutura, como energia elétrica, refrigeração, espaço físico e cabeamento de rede.

Arquiteturas do Núcleo

- **Máquina Virtual**
  - Aumento da disponibilidade
    - Uma vez que cada VM pode ser copiada facilmente para a memória secundária, como um disco, caso haja algum problema com uma VM ou com o próprio sistema onde está sendo processada, basta restaurar a cópia da VM em outro servidor e rapidamente o ambiente pode ser novamente disponibilizado.

Arquiteturas do Núcleo

- **Máquina Virtual**
  - <u>Facilidade de escalabilidade e balanceamento de carga</u>
    - Caso o sistema no qual a VM está sendo processada fique sobrecarregado e apresente problemas de desempenho, a VM pode ser facilmente migrada para um novo ambiente que possua maior capacidade de processamento. Dessa forma, é possível balancear a carga dos servidores e garantir o bom desempenho das aplicações.

Arquiteturas do Núcleo

- **Máquina Virtual**
  - Facilidade no desenvolvimento de *software*
    - As VMs permitem a criação de um ambiente independente para o desenvolvimento e teste de *software* sem o comprometimento das máquinas de produção. Além disso, é possível o teste do software em diferentes sistemas operacionais e suas versões sem a necessidade de um *hardware* dedicado.

Arquiteturas do Núcleo

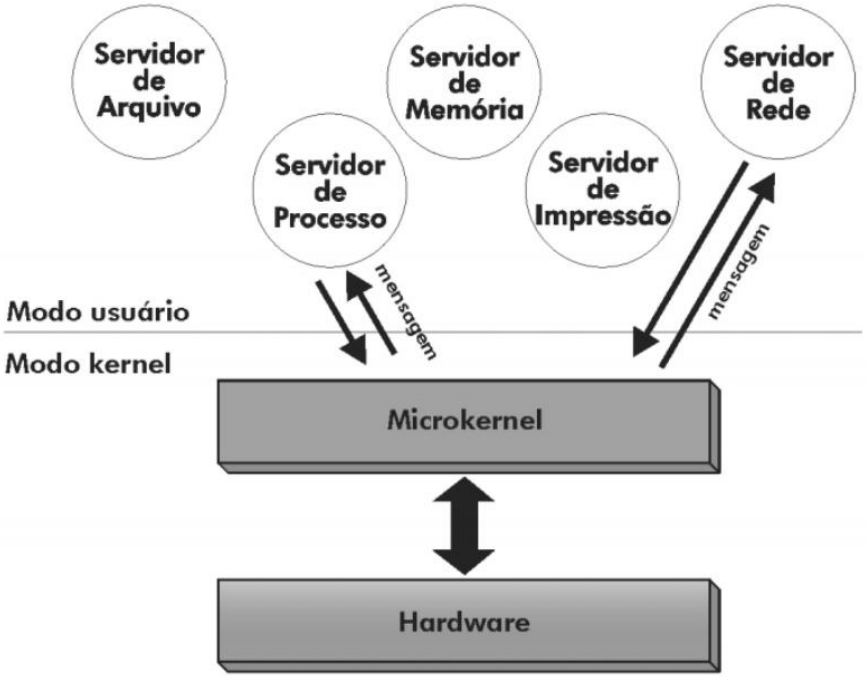
- **Arquitetura *Microkernel***
  - Uma tendência nos sistemas operacionais modernos é tornar o núcleo do sistema operacional o menor e mais simples possível. Para implementar esta ideia, os serviços do sistema são disponibilizados através de processos, onde cada um é responsável por oferecer um conjunto específico de funções, como gerência de arquivos, gerência de processos, gerência de memória e escalonamento.

  - Sempre que uma aplicação deseja algum serviço, é realizada uma solicitação ao processo responsável. Neste caso, a aplicação que solicita o serviço é chamada de cliente, enquanto o processo que responde à solicitação é chamado de servidor.

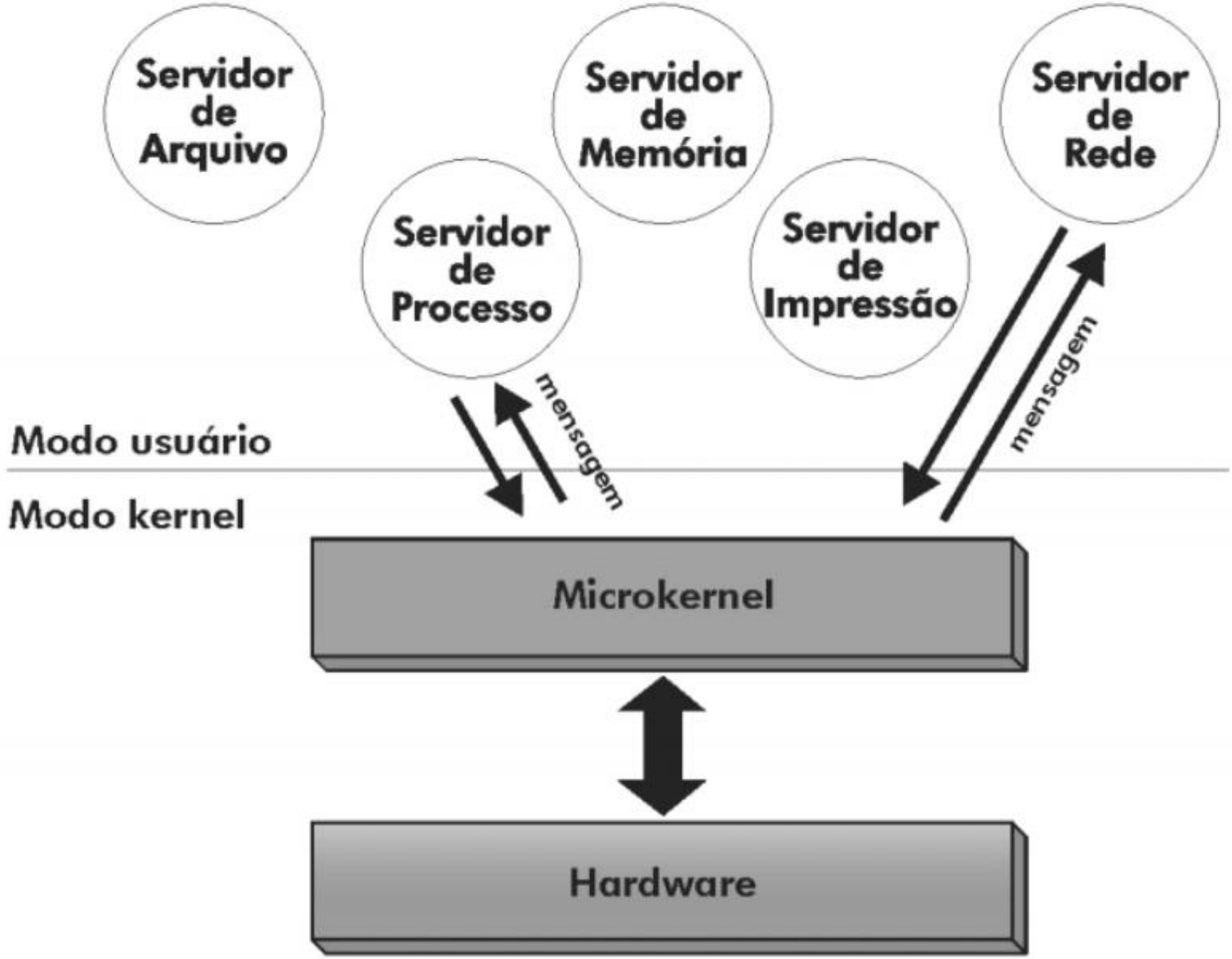
Arquiteturas do Núcleo

- **Arquitetura *Microkernel***
  - Um cliente, que pode ser uma aplicação de um usuário ou um outro componente do SO, solicita um serviço enviando uma mensagem para o servidor. O servidor responde ao cliente através de uma outra mensagem. A principal função do núcleo é realizar a comunicação, ou seja, a troca de mensagens entre cliente e servidor (ver Figura 10).

Figura 10 – Arquitetura *microkernel* genérica.



Fonte: (MACHADO; MAIA, 2013).


Servidor de Arquivo
Servidor de Memória
Servidor de Rede
Servidor de Processo
Servidor de Impressão
mensagem
mensagem
Modo usuário
Modo kernel
Microkernel
Hardware

Arquiteturas do Núcleo

- **Arquitetura *Microkernel***
  - O conceito da arquitetura *microkernel* surgiu no sistema operacional *Mach*, na década de 1980, na Universidade *Carnegie-Mellon*. O núcleo do sistema *Mach* oferece basicamente quatro serviços: gerência de processos, gerência de memória, comunicação por troca de mensagens e operações de E/S, todos em modo usuário.

  - A utilização deste modelo permite que os servidores executem em modo usuário, ou seja, não tenham acesso direto a certos componentes do sistema. Apenas o núcleo do sistema, responsável pela comunicação entre clientes e servidores, executa no modo *kernel*. Como consequência, se ocorrer um erro em um servidor, este poderá parar, mas o sistema não ficará inteiramente comprometido, aumentando assim a sua disponibilidade.

Arquiteturas do Núcleo

- **Arquitetura *Microkernel***
  - Como os servidores se comunicam através de trocas de mensagens, não importa se os clientes e servidores são processados em um sistema com um único processador, com múltiplos processadores (fortemente acoplado) ou ainda em um ambiente de sistema distribuído (fracamente acoplado). A implementação de sistemas *microkernel* em ambientes distribuídos permite que um cliente solicite um serviço e a resposta seja processada remotamente. Esta característica permite acrescentar novos servidores à medida que o número de clientes aumenta, conferindo uma grande escalabilidade ao SO.

  - Além disso, esta arquitetura permite isolar as funções do sistema operacional por diversos processos servidores pequenos e dedicados a serviços específicos, tornando o núcleo menor, mais fácil de depurar e, consequentemente, aumentando sua confiabilidade. Na arquitetura *microkernel*, o SO passa a ser de mais fácil manutenção, flexível e de maior portabilidade.

Arquiteturas do Núcleo

- **Arquitetura de Camadas**
  - Curiosidade
    - Apesar de todas as vantagens deste modelo, sua implementação, na prática, é muito difícil. Primeiro existe o problema de desempenho, devido à necessidade de mudança de modo de acesso a cada comunicação entre clientes e servidores. Outro problema é que certas funções do sistema operacional exigem acesso direto ao *hardware*, como operações de E/S. Na realidade, o que é implementado mais usualmente é uma combinação do modelo de camadas com a arquitetura *microkernel*. O núcleo do sistema, além de ser responsável pela comunicação entre cliente e servidor, passa a incorporar outras funções críticas do sistema, como escalonamento, tratamento de interrupções e gerência de dispositivos.

Referências

- MACHADO, Francis Berenger; MAIA, Luiz Paulo. **Arquitetura de sistemas operacionais**. LTC, 2013.

- TANENBAUM, Andrew S.; WOODHULL, Albert S. **Sistemas Operacionais: Projetjos e Implementação**. Bookman Editora, 2009.

- SILBERSCHATZ, Abraham. **Sistemas operacionais com Java**. Elsevier Brasil, 2008.